

Assignment: Restaurant Simulation with Threads and Synchronization

Objective

Build a simplified restaurant simulation using threads for the kitchen, waiters, and clients running concurrently. Practice: creating/running threads, identifying shared resources, protecting them with locks (mutex), and using `sleep()` to simulate time so you can observe how threads interleave in the output.

Scenario

Role	Count	Responsibility
Kitchen	1	Cooks one order at a time; protect its order queue with a lock.
Waiters	2	Take orders, deliver food; share access to the kitchen queue.
Clients	5	Arrive, order, wait, eat, leave — each runs concurrently.

Flow per client: arrive → assigned to a free waiter → waiter sends order to kitchen queue → kitchen cooks (sleep) → waiter delivers food → client eats (sleep) → leaves.

Requirements

- Protect with a lock: the kitchen's order queue, the pool of free waiters, and any shared counter/log.
- Kitchen, each waiter, and each of the 5 clients run as separate threads, started so their work overlaps.
- Use `sleep()` (varied, e.g. random 2–5s) to simulate cooking, serving, and eating time.
- Every printed line must include a timestamp and the actor's name, e.g. `[00:00:07] Kitchen: order for Client-3 is ready` — so interleaving and lock behavior are visible.

Deliverables

- Commented source code implementing the simulation.
- Short written analysis (½–1 page): which resources you locked and why; a snippet of your real output explained; what happens (with evidence) if you remove the kitchen-queue lock.
- Console output for one full run (all 5 clients served).

Using AI Tools (ChatGPT / Claude)

You may use an AI assistant to generate a first version of the code, but you must read, understand, and be able to explain every part of it — especially where locks are acquired/released and what would break without them. Your written analysis must be in your own words and reference your own program's actual output. Submitting code or analysis you can't explain is an academic integrity violation.

Grading Rubric (10 pts)

Criteria	Pts
Threads for kitchen, waiters, clients	2
Locks correctly protect shared resources	2
<code>sleep()</code> used to simulate cook/serve/eat time	1
Correct overall program logic/flow	1
Clear, timestamped console output	1
Written analysis (own words, own output)	3